git - 列印出

Git 速查表

入門

建立一個新的程式碼倉庫：
`git init`

克隆現有倉庫：
`git 克隆`

準備投入

新增未追蹤的檔案或未暫存的變更：
`git 新增`

新增所有未追蹤的檔案和未暫存的變更：git add .

選擇要暫存的文件部分：
`git add -p`

移動文件：
`git mv`

刪除檔案：
`git rm`

告訴 Git 忘記某個文件，但不要刪除它：
`git rm --cached`

取消暫存一個文件：
`git 重置`

取消一切舞台表演：
`git 重置`

檢查您新增的內容：
`git 狀態`

提交

提交更改（並打開文字編輯器寫入更改資訊）：

`git 提交`

提交一次請求：
`git commit -m 'message'`

提交所有未暫存的變更：
`git commit -am 'message'`

在分支間移動

切換分支：
`git 開關`

或者

`git checkout`

建立分支：
`git switch -c`

或者

`git checkout -b`

列出分公司：
`git 分支`

按最近提交次數列出分支：
`git branch --sort=-committerdate`

刪除分支：
`git 分支 -d`

強制刪除分支：
`git 分支 -D`

階段性/非階段性變更的差異

比較所有已暫存和未暫存的變更：
`git diff HEAD`

差異僅包含階段性變更：
`git diff --staged`

僅未暫存變更的差異：
`git diff`

差異提交

顯示提交與其父提交之間的差異：
`git show`

比較兩個提交：
`git diff`

與上次提交相比，單一文件的差異：
`git diff`

顯示差異摘要：
`git diff --stat`

`git show --stat`

引用提交的方式

每次我們說您可以使用以下任何一種方法：

- 主分支
- 標籤 v0.1
- 提交 ID 3e887ab
- 遠端分支 origin/main
- 目前提交 HEAD
- 3 次提交前 HEAD^^^ 或 HEAD~3

放棄您的更改

刪除對某個檔案的未暫存變更：
`git 復原`

或者

`git checkout`

刪除單一檔案的所有暫存和未暫存變更：

`git restore --staged --worktree`

或者

`git checkout HEAD`

刪除所有已暫存和未暫存的變更：
`git reset --hard`

刪除未追蹤的檔案：
`git clean`

“Stash”所有已安排和未安排的變更：
`git stash`

編輯歷史記錄

撤銷最近一次提交（保持工作目錄不變）：

`git reset HEAD^`

將最後 5 個承諾合併為一個：
`git rebase -i HEAD~6`

然後將“pick”改為“fixup”，以便將任何要與前一個提交合併的提交合併。撤銷失敗的變基：git reflog BRANCHNAME

然後手動在 reflog 中找到正確的提交 ID，然後執行：

`git reset --hard`

修改提交資訊（或新增您忘記新增的文件）：

`git commit --amend`

代碼考古

查看分支的歷史記錄：
`git log main`

`git log --graph main`

`git log --oneline`

顯示所有修改過文件的提交記錄：
`git 日誌`

顯示修改過該文件的所有提交記錄，包括重新命名之前的提交記錄：
`git log --follow`

找出所有新增或刪除文字的提交：

`git log -G banana`

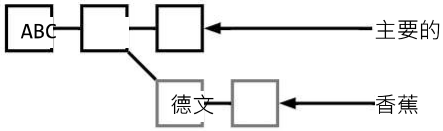
顯示文件中每一行的最後修改者：
`git blame`

合併分叉分支

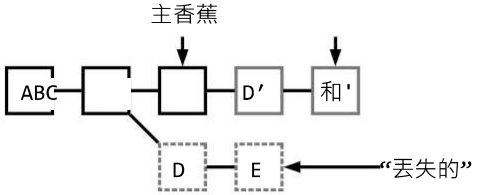
與 rebase 結合使用：

```
git 切換香蕉
git rebase main
```

前：



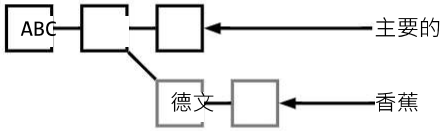
後：



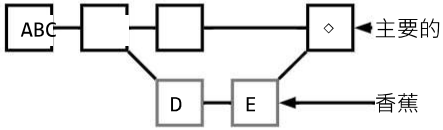
合併：

```
git 切換到主分支
git merge banana
```

前：



後：

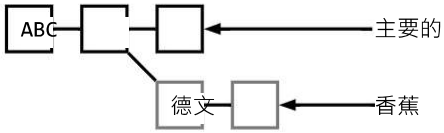


與 squash merge 合併：

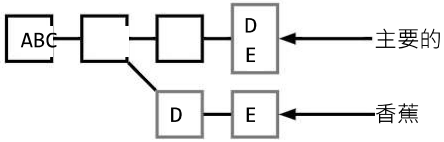
```
git switch main git merge --squash banana
```

```
git 提交
```

前：



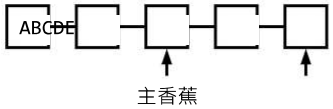
後：



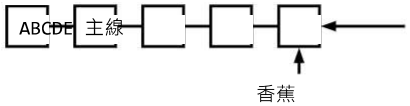
將一個分支與另一個分支同步更新（又稱「快速合併」）：

```
git 切換到主分支
git merge banana
```

前：



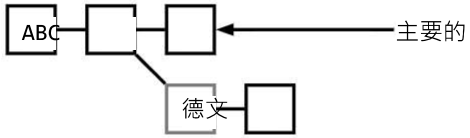
後：



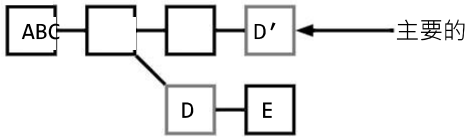
將一個提交複製到目前分支：

```
git cherry-pick
```

前：



後：



恢復舊文件

取得另一個提交中的文件版本：

```
git checkout
```

或者

```
git 復原 - 來源
```

新增遙控器

```
git 遠端新增
```

提交您的更改

推
主要的
分支到遠端
起源
：

```
git push origin main
```

將當前分支推送到其遠端「追蹤分支」：

```
git push
```

推一下你以前從未推過的樹枝：

```
git push -u origin
```

用力推：

```
git push --force-with-lease
```

推送標籤：

```
git push --tags
```

拉取更改

獲取更改（但不要更改任何本地分支）：

```
git fetch origin main
```

取得更改，然後對目前分支進行變基：

```
git pull --rebase
```

取得變更並將其合併到目前分支：

```
git pull origin main
```

或者

```
git pull
```

配置 Git

設定配置選項：

```
git config user.name '你的名字'
```

全域設定選項：

```
git config --global ...
```

新增別名：

```
git 設定別名.st 狀態
```

查看所有可能的配置選項：

```
git-config 手冊頁
```

重要文件

本地 Git 設定：

```
.git/config
```

全域 Git 配置：

```
~/.gitconfig
```

要忽略的文件列表：

```
.gitignore
```